

AF: Redundancy-Free Adaptive Dense Subgraph Mining

Overview

Graph mining is fundamental to numerous domains, including social network analysis, bioinformatics, and cybersecurity. Dense subgraph mining identifies maximal subgraphs that meet a structural requirement and is useful for tasks like community detection. Well-studied problems include maximal clique, quasi-clique, k -plex, and k -truss mining, typically solved with the Bron–Kerbosch (BK) algorithm, which explores the solution space by traversing a set-enumeration tree. However, this approach struggles to scale due to the exponential growth of the search space. Research has focused on pruning rules to reduce the candidate subgraphs explored.

Despite these efforts, redundancy remains a major challenge in BK-based search. For example, consider a graph with vertices $\{a, b, c, d, e\}$. If the algorithm identifies a valid k -plex $\{a, b, c, d\}$, it must still evaluate $\{b, c, d\}$ in order to eventually explore $\{b, c, d, e\}$. This intermediate step is redundant: $\{b, c, d\}$ itself cannot be maximal because $\{a, b, c, d\}$ already qualifies. Nonetheless, the BK framework requires this check, leading to wasted computation. These redundancies are pervasive in dense subgraph mining and become increasingly severe in large-scale graphs, where much of the runtime is consumed by such unnecessary exploration.

This project proposes a new search strategy to eliminate these inefficiencies. We will develop a tree re-ordering strategy to prevent revisiting subsets once a branch is fully enumerated. Additionally, we will explore adaptive search directions, considering bottom-up enumeration alongside the top-down approach, guided by a machine learning strategy. Finally, we will integrate these techniques into new algorithms for maximal subgraph mining, which requires redesigning pruning rules tailored to each problem.

Intellectual Merit

Dense subgraph mining is crucial in fields like social network analysis, bioinformatics, and cybersecurity. However, existing methods, such as clique, quasi-clique, k -plex, and k -truss mining, built on the BK framework, suffer from redundancy. As graphs grow, this redundancy increases and causes inefficiency. This project introduces a redundancy-free search paradigm. It integrates three innovations: (1) a tree re-ordering strategy to prevent redundant subset generation, (2) a bottom-up adaptive search strategy for pruning, and (3) algorithm-specific pruning rule redesigns for different dense subgraph problems. These innovations establish a new framework for efficient dense subgraph mining, offering a theoretical model for redundancy elimination and scalable algorithms. It has the potential to reshape graph mining, enabling practical analysis of large networks.

Broader Impacts

This research will significantly advance the analysis of large, complex networks, with direct applications in cybersecurity (e.g., anomaly detection), bioinformatics (e.g., protein interactions and gene analysis), and social sciences (e.g., community detection). The project will provide students from diverse backgrounds with hands-on experience in algorithm design, parallel computing, and data analytics, preparing them for careers in academia and industry. All developed algorithms and software will be made open-source, promoting transparency, reproducibility, and fostering collaboration across the research community. Moreover, the project will integrate innovative research methods into the *Advanced Design and Analysis of Algorithms* course, enriching the curriculum and exposing students to state-of-the-art technologies. By advancing graph algorithm knowledge and cultivating a diverse group of future computing professionals, the project will contribute to the growth of both the field and the next generation of innovators.

Project Description

1 Project Background and Motivation

Graph Mining. Graphs are effective for capturing complex relationships between entities and have been widely used to model real-world datasets. Graph mining is the process of discovering and extracting meaningful patterns, insights, and knowledge from graph data structures [1]. For example, one type of graph mining problem is the identification of dense subgraphs in a large graph [2, 3], which has important applications in community detection [4] and biological network analysis [5, 6]. One commonly used solution to this problem is the Bron–Kerbosch (BK) algorithm [7], a recursive backtracking approach that explores combinations of vertices using a set-enumeration tree as Figure 1. To improve efficiency in dense subgraph mining, various pruning strategies have been proposed to eliminate unpromising vertices early, thereby reducing the search space. In the following, we categorize graph mining problems based on their computational complexity.

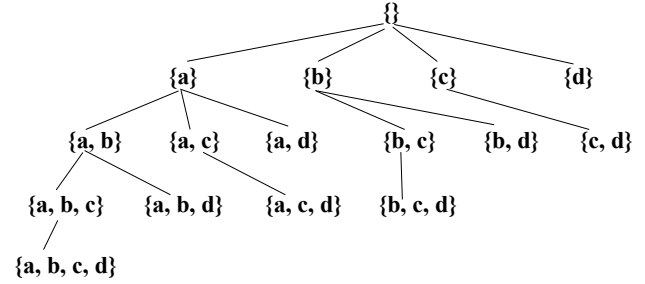


Figure 1: Set-Enumeration Tree

Low-Complexity Iterative Graph Algorithms. The first category is *iterative graph algorithms* that exhibit lower time complexity, often scaling as $O(m)$ or $O(m \cdot \text{polylog}(n))$, where n and m denote the numbers of vertices and edges. These algorithms work by propagating messages along edges across a small number of iterations. An example is *PageRank*, which simulates a random walk over a graph and converges quickly in practice. Despite their importance, these algorithms are usually **communication-intensive** rather than compute-intensive, and are less likely to benefit significantly from new search paradigm. Hence, they are not the focus of this proposal.

High-Complexity Compute-Intensive Graph Mining. The focus of this proposal is on compute-intensive graph mining problems, many of which are NP-hard. These problems present significant computational challenges and offer promising opportunities for leveraging new search paradigm. Dense subgraph mining problems are representative of compute-intensive tasks. These problems involve discovering subgraphs within a large input graph that meet specific structural constraints, such as density, connectivity, or degree bounds. Examples include *k-clique* [8, 9], *k-core* [10, 11], and *k-plex detection* [12, 13]. These problems are widely applicable, for instance in detecting communities in social networks or complexes in protein interaction networks, and are known to be NP-hard. They typically involve an exponential search space.

Dense Subgraph Mining Example. Dense subgraph mining tasks, such as maximal clique mining, quasi-clique enumeration, and *k-plex* mining, are critical for data-driven discovery in fields like bioinformatics, chemistry, and cybersecurity. These problems are NP-hard and often rely on traversing a set enumeration tree (SET) as Figure 1 to exhaustively explore candidate solutions. The SET represents the exponential combinatorial space of vertex subsets, with each node corresponds to a candidate subgraph.

For example, a *maximal clique* is a set of vertices that forms a complete subgraph and cannot be extended by including any other vertex. In maximal clique mining, the SET begins with the empty set and recursively adds vertices that preserve the clique condition.

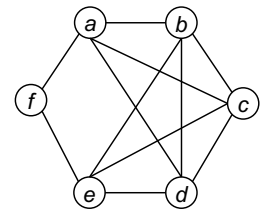


Figure 2: Maximal Clique

Consider a graph with vertex set $\{a, b, c, d, e, f\}$ as shown in Figure 2. The algorithm may explore the following branches in the SET and find the maximal cliques:

$\{\} \rightarrow \{a\} \rightarrow \{a, b\} \rightarrow \{a, b, c\} \rightarrow \{a, b, c, d\}$: a maximal clique of size 4;

$\{\} \rightarrow \{b\} \rightarrow \{b, c\} \rightarrow \{b, c, d\} \rightarrow \{b, c, d, e\}$: another maximal clique of size 4.

Classical algorithms must traverse a combinatorial search space that, in the worst case, approaches the power set of the vertex set. Even with aggressive pruning strategies, the search space remains exponential. Moreover, verifying whether a candidate subset forms a clique requires ensuring that each vertex is adjacent to all others in the set—a process that, while polynomial, incurs nontrivial overhead as the number of candidates increases. The inherent difficulty arises from the need to enumerate and compare an enormous number of subgraph candidates. Despite effective pruning strategies, these problems remain NP-hard and computationally expensive.

Current approaches for dense subgraph mining rely on the BK algorithm, which exhibits significant inefficiencies. The primary sources of these inefficiencies in the BK framework are twofold:

1. **Non-maximal subset check redundancy:** In a standard BK search tree, intermediate subsets must be checked even if they are non-maximal and cannot be part of a final solution. For example, consider the graph in Figure 2 and the corresponding search tree in Figure 1. After identifying the quasi-clique $\{a, b, c, d\}$, the algorithm still checks subsets such as $\{b, c\}$, $\{b, d\}$, $\{b, c, d\}$, and $\{c, d\}$. Even though these subsets are already known to be non-maximal, the BK framework requires evaluating them to eventually reach the qualified output $\{b, c, d, e\}$. This redundant checking consumes significant computational resources, particularly in large graphs where such intermediate steps are numerous.
2. **Search path inefficiency:** The top-down enumeration of BK does not exploit the potential efficiency of a bottom-up search. Reasoning backward from the maximal results, it is clear that one can find the qualified maximal subgraphs more efficiently by starting from the full vertex set $\{a, b, c, d, e\}$ and iteratively removing vertices. This bottom-up approach can quickly identify the maximal results $\{a, b, c, d\}$ and $\{b, c, d, e\}$ without checking all intermediate subsets, thereby reducing the search space dramatically. However, determining when to switch between bottom-up and top-down search is challenging, as it depends on both the specific algorithm and the dataset structure.

These two inefficiencies, redundant subset checks and inflexible search paths, highlight the need for a **new search paradigm** that minimizes wasted computation while maintaining correctness. This motivates our proposed approach: a redundancy-free search strategy that combines tree re-ordering to avoid revisiting subsets with an adaptive search direction that dynamically chooses between top-down and bottom-up enumeration. By addressing these fundamental inefficiencies, our project aims to establish a more scalable and efficient framework for dense subgraph mining across a variety of graph mining problems.

Intellectual Merit. These inefficiencies fundamentally limit scalability and prevent the practical analysis of large, real-world networks. Addressing this limitation requires not just improved pruning heuristics, but a redesign of the underlying set-enumeration search strategy. This project introduces a redundancy-free framework for maximal subgraph mining that integrates three transformative innovations:

1. **Tree Re-ordering Strategy:** We will design a re-ordering mechanism in the set-enumeration tree. Once a maximal subset is identified, vertices in the qualified subset will be reordered to the end while unchecked vertices move forward, ensuring that non-maximal subsets are not redundantly revisited.
2. **Adaptive Bidirectional Search:** Beyond the conventional top-down approach, we will enable bottom-up enumeration that begins with the full vertex set and removes vertices iteratively. A machine learning model will dynamically choose between top-down, bottom-up, or hybrid strategies, guided by graph structure and runtime feedback. Furthermore, we will augment depth-first search with the binary search approach (batch expansion), adding or removing multiple vertices at once to accelerate convergence toward qualified subsets.
3. **Integration with Pruning Rule Redesign:** The new traversal strategies will be integrated into algorithms for dense subgraph problems, including maximal clique mining, quasi-clique enumeration, and k -plex mining. Because pruning rules are tightly coupled with traversal order, this project will redesign problem-specific pruning strategies tailored to redundancy-free search. This redesign is non-trivial, requiring theoretical reformulation of correctness guarantees and efficiency bounds.

Collectively, these innovations establish a new class of **redundancy-free set-enumeration algorithms** that go beyond incremental pruning and directly eliminate wasted computation. The project’s intellectual contributions include:

- A formal theoretical model of redundancy-free subgraph exploration;
- Adaptive algorithmic frameworks that unify top-down, bottom-up, and binary search in graph traversal;
- Newly tailored pruning rules under the new traversal paradigm for each dense subgraph mining problems, yielding provably more efficient solutions.

By advancing both the theoretical foundations and algorithmic techniques of dense subgraph mining, this work has the potential to reshape graph mining as a discipline, making previously intractable problems feasible on large-scale networks.

Preparedness and PI’s qualification. The PI has extensive experience in building high-performance systems and designing scalable algorithms for large-scale graph mining. To accelerate the NP-hard graph mining algorithm, the PI has developed a comprehensive parallel framework of systems, collectively called G-thinker [14–16]. Upon this framework, the PI has parallelized a wide range of NP-hard graph mining algorithms, including maximum clique finding [14], maximal quasi-clique mining [17–19], frequent pattern mining [20,21], subgraph matching [14]. These systems have achieved performance improvements of up to several orders of magnitude compared to competing approaches. Through these research efforts, the PI has developed deep insight into the structural and computational properties of graph mining problems, along with expertise in decomposing complex algorithms for parallel execution. These skills are highly transferable to the quantum setting, where circuit-level algorithm optimization is similarly critical. The PI’s work has been published in premier venues, including PVLDB, VLDBJ, ICDE, PPoPP, and TKDE [14–17,19–23], and has been recognized with invited tutorials [24] and book chapters [5].

With the advancement of GPUs and the increasing demonstration of their computational power, this PI has broken new ground by developing scalable GPU solutions to the various compute-intensive graph problems. Supported by the PI’s NSF CRII award [25], several GPU-based graph

mining systems have been developed to accelerate a range of algorithms, including k -core decomposition [26] and community search [23]. In addition, two manuscripts on GPU-accelerated maximal quasi-clique mining and graph query processing are currently under review at CIKM 2025.

2 Preliminary Result

2.1 Limitation of Parallel Solutions

how many unnecessary check happened

2.2 Initial Example on Maximal Clique Mining

what is Maximal Clique Mining Do it in

3 Research Plan

This research aims to eliminate the redundancy in current set-enumeration search tree, which is followed by most of the current dense subgraph mining algorithms. This project will first explore an re-ordering strategy to avoid the enumeration on the unpromising step in current searching tree. Then it will hybrid the top-down and bottom-up search strategy to earlier reach the maximal results in dense subgraph mining. Based on the above two new search schema, we will re-design the algorithms for multiple dense subgraph mining algorithms, including re-think the new compatible pruning techniques for new search pattern which is the most powerful approach to scale to large graphs in current implementation.

3.1 Thrust 1: Re-ordering Tree Search.

Figure 4 show an example when mining a maximal clique in graph (as Figure 3) has vertices $\{a, b, c, d\}$. The current solution are searching the set-enumeration tree as the left in Figure 4 following the BK algorithm's workflow. The qualified maximal cliques are underlined in green. As it shown, it will do the Depth-First Search traversal and find the $\{a, b, c\}$ is a maximal clique as it cannot expand to $\{a, b, c, d\}$ to form a larger clique. Then it need to trace back. The problem in the current enumeration approach is that it need to check some redundant combination, as enclosed by a dashed line. But the current algorithms cannot bypass those redundant steps to find all the valid results. For example, to find the qualified result $\{a, c, d\}$, it must go through the $\{a, c\}$, even it is unpromising, as it must be covered by the $\{a, b, c\}$ we already found in previous steps.

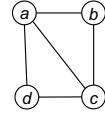


Figure 3: Maximal Clique Example

Not only in maximal clique mining, we noticed this redundancy is common problem in most our current dense subgraph mining. We noticed these problem stems from that we cannot make full use of the previous results. Just like the example in Figure 4, we find the qualified result $\{a, b, c\}$ but cannot use this information to optimize the later mining step, since we are trying to enumerate the maximal results.

Also, this redundancy is huge and can account most of current mining. If we already find a set is qualified, theoretically, all its subsets are not maximal and can be skipped (if not visited yet). As the example in Figure 4, the subset $\{a, c\}$, $\{b\}$, $\{b, c\}$ and $\{c\}$ are all redundancy. This is just a simple example, with 3 vertex qualified. Theoretically, the steps can be saved is 2^n , as a set with n elements has 2^n subsets in total, with exactly $\binom{n}{k}$ subsets of size k .

To save the redundancy steps using the found results in dense subgraph mining, in thrust 1, we will design a set enumeration tree re-ordering approach. Like shown on the right search tree in Figure 4, after found the qualified result $\{a, b, c\}$, what we need to do is find all the result must have $\{d\}$. From this perspective, why don't we just expand the rest tree start with $\{d\}$. So in the new search tree, we will revise the index by putting $\{d\}$ ahead and leave the $\{a, b, c\}$ to the tail. Also, we will only need to explore the sub-tree spawned from $\{d\}$. As shown in Figure 4 right side, we will save the four redundancy in this simple example.

Recursive Re-Ordering Strategy. The power of this re-ordering approach as it can be called recursively in each sub-tree. As shown in Figure 4, we found the qualified result $\{d, a, c\}$. Since the origin or the goal of the problem is to find result must including $\{d\}$, but we found $\{d, a, c\}$ qualified, that mean we can do a further re-ordering in the sub-tree starting from $\{d\}$ by: 1. putting $\{a, c\}$ to tail, 2. putting $\{b\}$ ahead to find all the qualified result which must including $\{d, b\}$. In this way, we only need to check $\{d, b, c\}$, which save the step of $\{d, c\}$???

todo: explain how to save $\{d, b, a\}$.

Example with 6 nodes. With the idea above, we want to show the power of this approach on

examples with more vertices to show the power to eliminate the redundancy and take full use the found information. As the example in Figure 2 ...

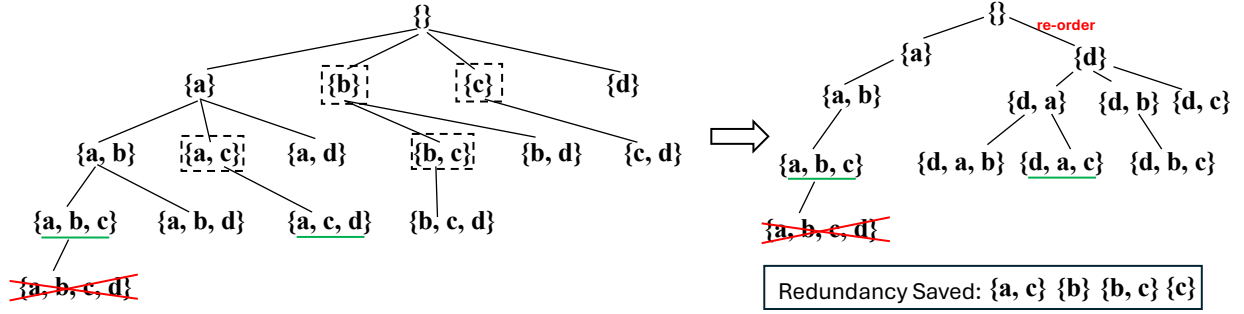


Figure 4: Redundancy-Saving Tree Search via Re-ordering Strategies

3.2 Thrust 2: Top-Down and Bottom-Up Hybrid Enumeration

The bottom-up strategy explores the search space by recursively removing one vertex at a time. To guarantee uniqueness and avoid redundant exploration, the removal order is constrained: the vertex eliminated at each step must be greater (e.g., in lexicographic order) than the vertex removed in the previous step. This monotonic ordering ensures that each candidate set is generated exactly once. A further benefit of the bottom-up strategy is its inherent pruning capability: once a valid set is identified, the anti-monotonicity property allows the entire subtree rooted at that set to be discarded. For instance, in Figure 5, once the sets $\{a, b, c\}$ and $\{a, c, d\}$ are discovered, their corresponding sub-trees can be safely pruned, thereby reducing redundant computations.

Binary Search over the Reversed Search Tree. The strength of this vertex-removal strategy lies in its pruning ability: once a qualified combination is identified, the entire subtree of its subsets can be safely discarded. To further accelerate the process, we propose applying a binary search over the reversed search tree (see Figure). When a qualified result is found, all candidate sets in the subtree beneath it can be eliminated. The maximal result must then reside in the portion of the tree above this qualified node. By traversing upward and downward in a binary search-like manner, we can efficiently pinpoint the maximal result. Rather than relying on a brute-force binary search, we will also develop efficient algorithms to derive tight upper and lower bounds, thereby guiding the binary search enumeration more effectively.

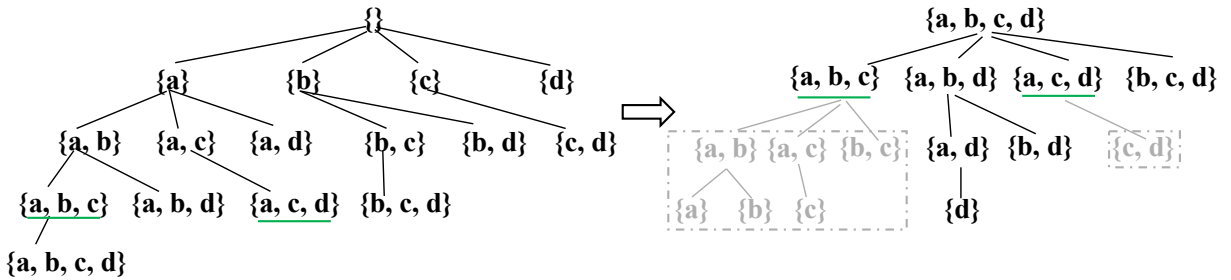


Figure 5: Bottom-Up Enumeration

Cooperated with Thrust 1 Re-Ordering Strategy. While Thrust 1 focuses on eliminating redundancy through dynamic re-ordering of the set-enumeration tree, Thrust 2 complements this idea by combining top-down and bottom-up exploration into a unified hybrid framework. The top-down strategy expands candidate sets by successively adding vertices, whereas the bottom-up strategy reduces candidate sets by systematically removing vertices. By interleaving these

two perspectives, the hybrid approach allows the algorithm to reach maximal results earlier and prune unpromising branches more aggressively. In particular, when guided by the re-ordering mechanism from Thrust 1, the hybrid enumeration ensures that both expansion and reduction steps leverage previously discovered results, avoiding repeated work and accelerating convergence to dense subgraphs. This cooperative design enables a tighter integration of pruning, re-ordering, and traversal order, providing a scalable path toward handling large and complex graphs.

3.3 Thrust 3: Re-designed Dense Subgraph Mining Algorithms.